

Application Structure

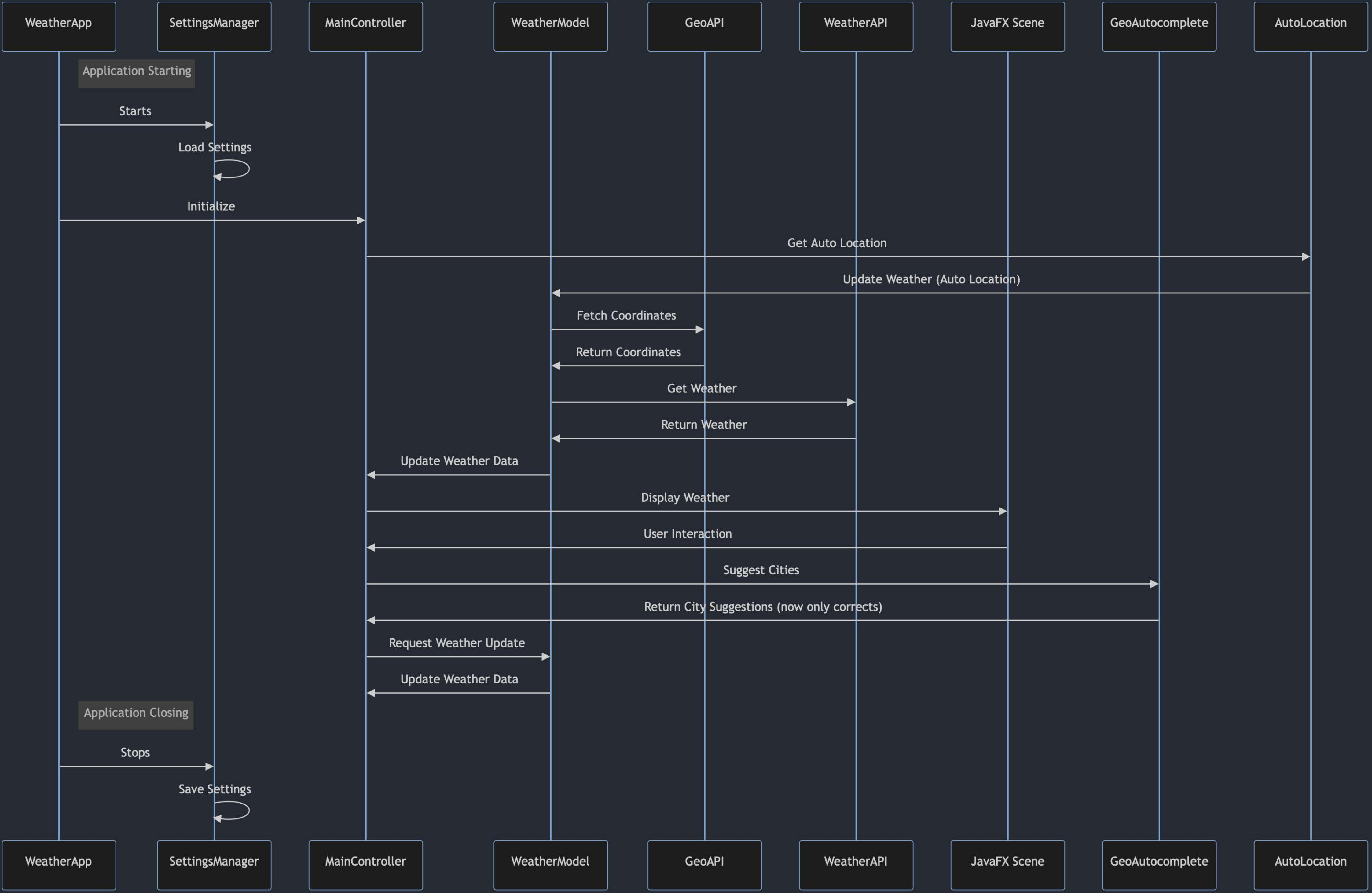
The application follows the Model-View-Controller (MVC) pattern, using JavaFX for the user interface:

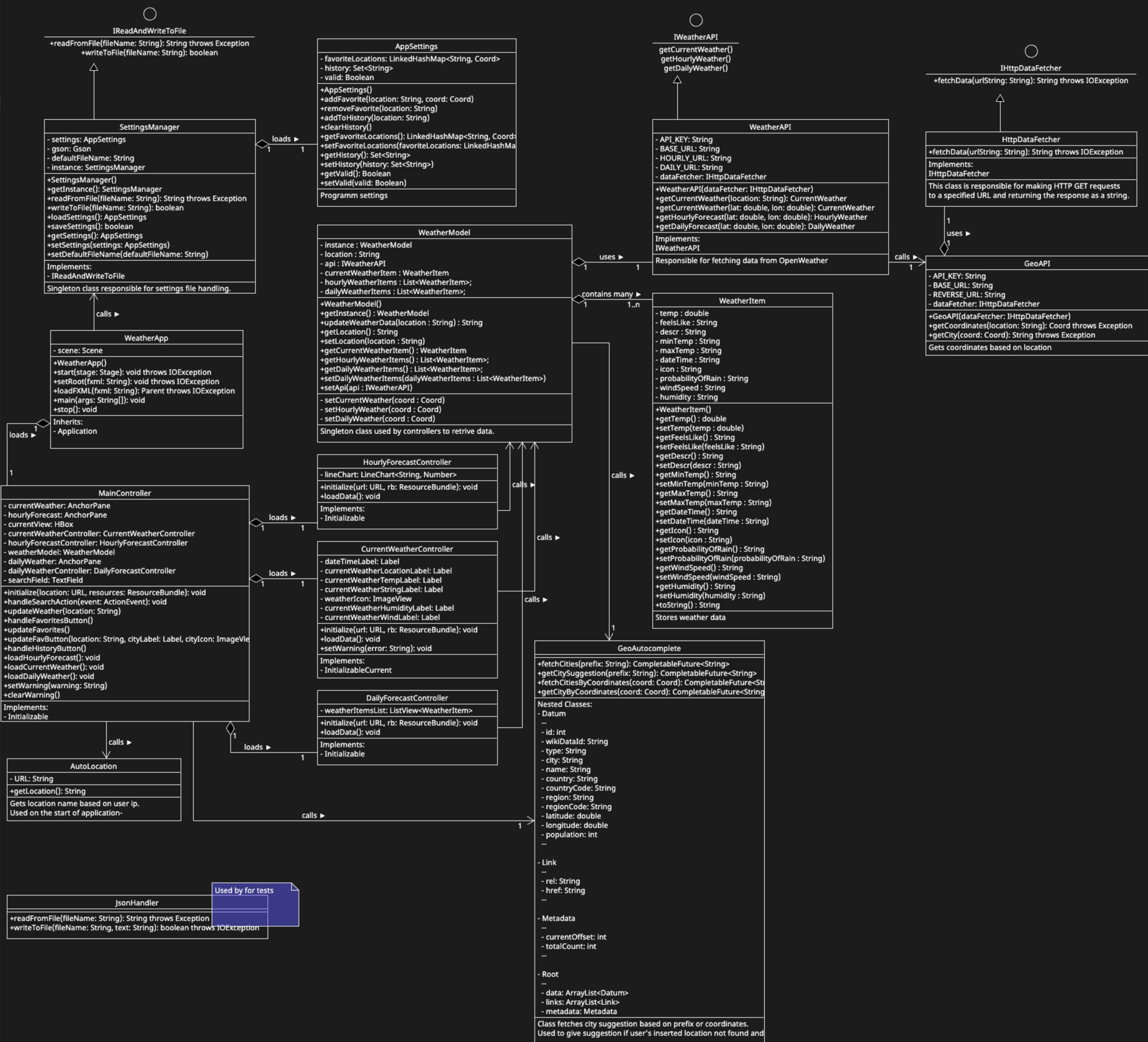
- **Model (`WeatherModel`)**: Manages the application's data and business logic.
- **View (`CurrentWeatherController` , `DailyForecastController` , `HourlyForecastController`)**: Renders the user interface and presents data to the user.
- **Controller**: Handles user input and translates it into actions to be performed by the model or view.

Key Responsibilities of Classes

- `WeatherApp` : The main entry point for the application, responsible for initializing the JavaFX framework.
- `WeatherModel` : A singleton class that holds weather data and is the primary data model for the application.
- `WeatherItem` : Represents individual weather data points or forecasts.
- `IWeatherAPI` : An interface to define how to retrieve weather information.
- `WeatherAPI` : Handles communication with the OpenWeatherMap API to retrieve weather information using OpenWeather API, implements `IWeatherAPI` .
- `SettingsManager` : Manages user settings file.
- `AppSettings` : Application settings, including user's favorite locations and search history.
- `GeoAPI` : Provides geolocation services by converting location names into geographical coordinates.
- `IHttpDataFetcher` : An interface that defines how HTTP requests are made, allowing for different implementations.
- `HttpDataFetcher` : Makes HTTP GET requests to a specified URL, implements `IHttpDataFetcher` .
- `GeoAutocomplete` : Provides methods for guessing geographical location names based on first letters, uses GeoDB API.
- `IReadAndWriteToFile` : Interface with methods to read from a file and write to a file.
- `JsonHandler` : Handles reading from and writing to JSON files, provides functionality to read the contents of a file and write text to a file. Used only in testing side of application.
- `AutoLocation` : Class which has method to retrieve the city location of the user based on their IP address, uses IP Geolocation API.

Responsibilities of controllers correspond to views they manipulate.





Examples of Pre- and Post-Conditions for some functions

- `WeatherModel.getInstance()` :
 - **Pre-Condition:** None.
 - **Post-Condition:** If no instance exists, one is created and returned. If an instance already exists, it is returned.
- `WeatherModel.updateWeatherData(String location)` :
 - **Pre-Condition:** The `location` parameter should be a valid location name that can be resolved to coordinates by `GeoAPI`.
 - **Post-Condition:** If successful, the current, hourly, and daily weather data are updated. Returns an empty string on success, or an error message if an error occurs.
- `WeatherModel.setCurrentWeather(Coord coord)` :
 - **Pre-Condition:** `coord` must contain valid latitude and longitude values.
 - **Post-Condition:** The `currentWeatherItem` is updated with the latest current weather data.
- `WeatherModel.setHourlyWeather(Coord coord)` :
 - **Pre-Condition:** `coord` must contain valid latitude and longitude values.
 - **Post-Condition:** The `hourlyWeatherItems` list is updated with the latest hourly weather data.
- `WeatherModel.setDailyWeather(Coord coord)` :
 - **Pre-Condition:** `coord` must contain valid latitude and longitude values.
 - **Post-Condition:** The `dailyWeatherItems` list is updated with the latest daily weather data.
- `WeatherAPI.getCurrentWeather(String location)` :
 - **Pre-Condition:** The `location` parameter must be a non-null, non-empty string that represents a valid location recognizable by the OpenWeatherMap API.
 - **Post-Condition:** Returns a `CurrentWeather` object populated with weather data for the given location. Throws an `IOException` if the data fetch operation fails.
- `WeatherAPI.getCurrentWeather(double lat, double lon)` :
 - **Pre-Condition:** The `lat` and `lon` parameters must be within the valid range for latitude (-90 to 90) and longitude (-180 to 180), respectively.
 - **Post-Condition:** Returns a `CurrentWeather` object populated with weather data for the given coordinates. Throws an `IOException` if the data fetch operation fails.
- `WeatherAPI.getHourlyForecast(double lat, double lon)` :
 - **Pre-Condition:** The `lat` and `lon` parameters must be within the valid range for latitude and longitude.
 - **Post-Condition:** Returns an `HourlyWeather` object containing the hourly weather forecast for the given coordinates. Throws an `IOException` if the data fetch operation fails.

- **WeatherAPI.getDailyForecast(double lat, double lon) :**
 - **Pre-Condition:** The `lat` and `lon` parameters must be within the valid range for latitude and longitude.
 - **Post-Condition:** Returns a `DailyWeather` object containing the daily weather forecast for the given coordinates. Throws an `IOException` if the data fetch operation fails.
- **GeoAutocomplete.getCitySuggestion(String prefix) :**
 - **Pre-Condition:** The `prefix` parameter must be a non-null, non-empty string.
 - **Post-Condition:** Returns a `CompletableFuture<String>` that, upon completion, provides the suggested city name. If no cities are found with the exact prefix, the method shortens the prefix and retries until suggestions are found. If an error occurs or no cities can be suggested even after shortening the prefix, the future completes with an error message or an empty string.
- **SettingsManager.getInstance() :**
 - **Pre-Condition:** None.
 - **Post-Condition:** Returns the existing instance of `SettingsManager` or creates a new one if it does not exist.
- **SettingsManager.readFromFile(String fileName) :**
 - **Pre-Condition:** The `fileName` parameter must be a non-null, non-empty string that corresponds to a readable file.
 - **Post-Condition:** Returns the file content as a string. Throws an exception if the file cannot be read.
- **SettingsManager.writeToFile(String fileName) :**
 - **Pre-Condition:** The `fileName` parameter must be a non-null, non-empty string. The `settings` object must be in a state that can be serialized to JSON.
 - **Post-Condition:** Writes the settings to the specified file and returns `true` if successful. If an error occurs, it returns `false`.
- **SettingsManager.loadSettings() :**
 - **Pre-Condition:** The default file name is set, and the file exists at the specified path.
 - **Post-Condition:** If the file exists and is readable, the settings are loaded into the `settings` object and `valid` is set to `true`. If the file does not exist or cannot be read, `valid` is set to `false` and default settings are created.
- **SettingsManager.saveSettings() :**
 - **Pre-Condition:** The `settings` object must be in a state that can be serialized to JSON.
 - **Post-Condition:** The settings are saved to the default file and the method returns `true` if the save operation is successful. If the save operation fails, it returns `false`.
- **SettingsManager.getSettings() :**
 - **Pre-Condition:** None.
 - **Post-Condition:** Returns the current `settings` object.

- **SettingsManager.setSettings(AppSettings settings) :**
 - **Pre-Condition:** The `settings` parameter must be a non-null `AppSettings` object.
 - **Post-Condition:** The internal `settings` object is updated to the provided `settings`.
- **SettingsManager.setDefaultFileName(String defaultFileName) :**
 - **Pre-Condition:** The `defaultFileName` parameter must be a non-null, non-empty string that represents a valid file name.
 - **Post-Condition:** The `defaultFileName` is updated to the provided string.

Work division and problems

Work division

Work in the team was mainly divided in terms of Views-Controllers and Models. However overall structure of the program have been discussed and agreed on together. Most of the problems and bugs have been solved using screen sharing.

Problems

Ivan has proposed an idea for views, that each separate weather instance (current, hourly, daily) will have its own fxml and controller. This approach now viewed as causing too many extra problems with data flow and especially interface (käyttöliittymä) design. Despite more lines of code were required for logic part of application, most of the efforts have been spent towards views.

Decent amount of time was spent trying to implement autocompletion feature for search ComboBox (initially TextField). We have been able to use GeoDB Cities API to get city suggestions based on first inserted letters, however implementing full feature did not succeed. According to research and our tries the correct approach is to use ControlsFX AutoCompletionTextFieldBinding, however some problems have been found related to Java version and modules, which we were not able to solve, although solution according to google was simple.

We also have been trying to avoid situation in which user types "helsinki" and it shows/saved as "helsinki", so city name was retrieved from openweather response. However, with further research it was found out that for some locations openweather api returns areas/districts, for example Amuri for Tampere. Geocoding API also did not provide relevant results, that's why GeoDB Cities API was used again, since there is an option to filter by city name.

Further improvements

Currently most of the functions are in try-catch block to prevent program crashing, although better approach would be to introduce more checks for input quality.

Autocompletion feature has to be introduced for search field.

All functions which fetch data have to be asynchronous, so UI will work smoothly.

UI has to be improved drastically.

Extra features implemented

- Chart is used to show hourly temperature
- When user gives wrongly spelled location, program tries to guess it. For example "Helsinki" will result in "Helsinki". Function decreases the length of the word to find first matching city. Initially it was supposed to be used for autocompletion.
- When program is opened it tries to guess current location of user (city). It contradicts with requirement regarding saving current weather location, but assumed to be more interesting.